

Chakravyuham: Behavior-Driven Edge AI Intrusion Detection for In-Vehicle Networks*

Pranav Raj Galipalli Addanki Srinidhi Tejasri Meedimale Abhinav Mucharla
Thota Bindu Darshitha

1 Executive Summary

Chakravyuham is a machine learning-based Intrusion Detection System (IDS) designed to detect and classify malicious activity on in-vehicle Controller Area Network (CAN) bus systems. Modern vehicles consist of interconnected Electronic Control Units (ECUs) that communicate over CAN, a protocol that inherently lacks authentication and encryption, making it vulnerable to message injection, spoofing, and denial-of-service attacks. While existing automotive cybersecurity solutions rely heavily on rule-based detection systems, these approaches struggle to identify subtle behavioral attacks and do not generalize well across different vehicle architectures.

Our project addresses this gap by developing a behavior-driven IDS using Graph Neural Networks (GCNs) and Gradient Boosted Decision Trees (XGBoost and LightGBM), trained on real-world datasets such as the HCRL Car-Hacking and OTIDS datasets. Instead of relying on raw identifiers or dataset-specific shortcuts, our system learns communication patterns through engineered behavioral features such as inter-arrival time, entropy, transition dynamics, and graph topology representations of CAN message flows.

During development, we identified that conventional evaluation practices in IDS research often overestimate model performance due to dataset leakage and artifact memorization. This led us to redesign our training and evaluation pipeline with temporal splits, block-wise partitioning, and feature dropout mechanisms to ensure realistic performance estimation and reduce shortcut learning.

Chakravyuham ultimately proposes a more rigorous and deployable approach to automotive intrusion detection by combining behavioral machine learning, explainable feature analysis, and a deployment-aware design philosophy aimed at bridging the gap between academic IDS models and real-world in-vehicle cybersecurity systems.

2 Threat Landscape & Problem Statement

Modern vehicles use the Controller Area Network (CAN) bus as the primary communication backbone between Electronic Control Units (ECUs) such as braking, engine control, infotainment, and steering systems. CAN was originally designed for reliability and efficiency in closed vehicle environments, not for security. As a result, it provides neither authentication nor encryption, meaning any node with bus access can inject or modify messages that other ECUs will blindly trust.

This fundamental design weakness makes CAN networks vulnerable to a range of real-world attacks, including Denial-of-Service (DoS) flooding, message spoofing, and fuzzy (random injection) attacks. In severe cases, attackers can impersonate critical ECUs and influence safety-critical functions such as braking or acceleration. This is not theoretical — the 2015 Jeep Cherokee remote exploitation demonstrated how external access to

*Submitted as a technical concept proposal for Tata Technologies InnoVent-27 (4th Edition) under the theme “AI at the Edge” in partnership with Amazon Web Services (AWS) and Emerson.

vehicle systems can escalate into direct control over physical vehicle behavior, highlighting the real-world consequences of CAN-level vulnerabilities.

To address these risks, automotive cybersecurity has become a regulated requirement under standards such as UN R155 and ISO/SAE 21434, pushing manufacturers to incorporate intrusion detection mechanisms within modern vehicles. However, current industrial solutions are still largely dominated by rule-based or signature-based Intrusion Detection Systems. While these approaches are efficient and certifiable for in-vehicle deployment, they rely on predefined patterns and struggle to detect unknown or evolving attack behaviors.

Machine learning-based IDS approaches offer the potential to detect subtle behavioral anomalies that rule-based systems may miss. However, their adoption in real automotive systems remains limited due to several critical challenges: lack of standardized data representation across OEMs, differences in ECU architectures and message timing across vehicles, and most importantly, the tendency of ML models to overfit dataset-specific artifacts rather than learning true attack behavior. Additionally, improper evaluation methodologies in existing research often lead to overly optimistic performance results that do not reflect real-world deployment conditions.

This creates a clear gap: while rule-based systems are deployable but limited in detection capability, machine learning systems show promise but lack reliability, generalization, and rigorous evaluation for real-world automotive deployment. Existing benchmark datasets primarily evaluate flooding, fuzzing, and spoofing-style attacks. More advanced temporal attacks such as replay attacks remain comparatively underexplored and typically require models capable of reasoning over long-term communication sequences.

Chakravyuham is built to address this gap by focusing on behavior-driven detection, leakage-resistant evaluation, and a deployment-aware ML pipeline designed specifically for realistic in-vehicle cybersecurity constraints.

3 Solution Overview / Core Architecture

3.1 Pipeline overview

Chakravyuham processes raw in-vehicle communication stream packets through a structured pipeline that abstracts low-level data frame formats into robust network-wide behavioral signatures, as illustrated in Figure 1.

3.1.1 1. CAN Data Ingestion

- Raw CAN traffic is collected from the HCRL Car-Hacking and OTIDS datasets.
- Each HCRL attack file is an **independent recording session** with its own timestamp origin.
- Sessions are processed independently to preserve chronological communication patterns.
- Files are **never shuffled together**, since they represent separate recordings on different clocks.
- Dataset combination occurs only after preprocessing/feature extraction where appropriate, preventing artificial temporal mixing.

3.1.2 2. Real-Time Packet-Level Classification and Windowing

- **Packet-Level Tabular Classification:** To achieve zero-lag, sub-millisecond intrusion detection, the primary GBDT models (LightGBM, XGBoost) operate directly at the **individual packet level**. They classify each incoming CAN message independently, requiring no window aggregation delay.
- **Neural Network Windowing:** Only the deep learning pathways (GCN and MosaicCNN) segment the CAN streams into sliding message-count windows ($N_W = 200$ for GCN, $N_W = 64$ for CNN) to construct relational graph topologies and spatial mosaic matrices.
- **Session-Aware Block-Wise Partitioning:** To prevent train-test leakage, training data is segmented into chronological 10-second temporal blocks. These blocks are shuffled and split, ensuring temporally adjacent packets do not appear in both train and test sets.

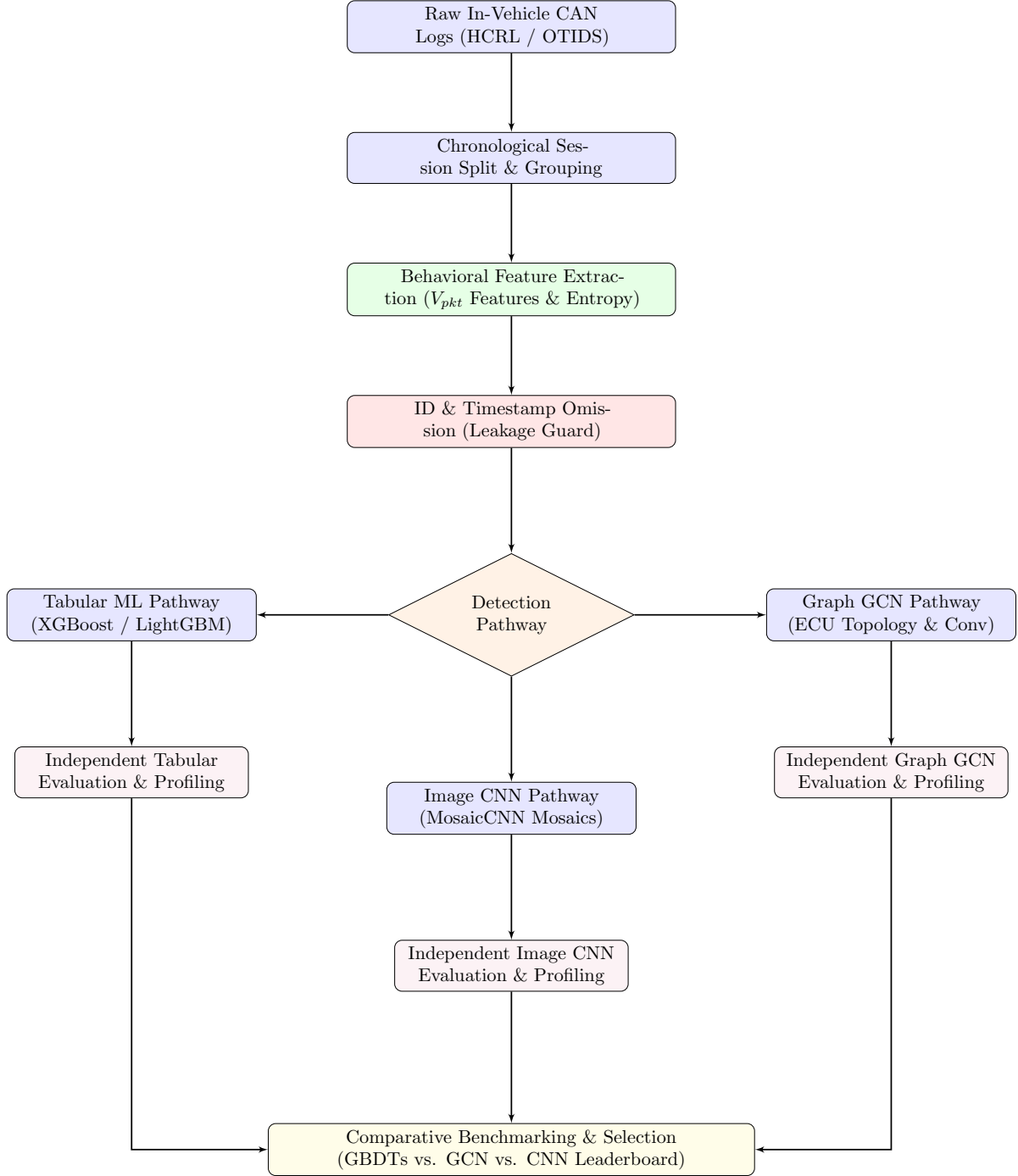


Figure 1: Chakravyuham Intrusion Detection System Data and Processing Pipeline

3.1.3 3. Behavioural Feature Extraction

Instead of relying on raw CAN identifiers, Chakravyuham represents each CAN message using a Unified Standard Feature Vector (V_{pkt}) containing timing and content statistics:

$$V_{pkt} = [IAT, Jitter, F_{rolling}, H_{entropy}, H_{distance}]$$

The feature vector encapsulates the following engineered behavioural signals:

Timing & Frequency Behaviour

- **Inter-Arrival Time (IAT)**: Computed as $\Delta t_k = t_k - t_{k-1}$ grouped by CAN ID. Measures the elapsed time between consecutive transmissions of the same arbitration ID, identifying volumetric anomalies such as DoS flooding.
- **Jitter**: Computed as $J_k = |\Delta t_k - \Delta t_{k-1}|$ grouped by CAN ID. Measures the timing variance between consecutive inter-arrival times of the same ID, detecting clock drift and spoofing injections.
- **Rolling Message Frequency ($F_{rolling}$)**: Computed as $F_k = \frac{50}{t_k - t_{k-50}}$ globally across the bus. Measures the global transmission rate over a rolling count of 50 messages to detect global bus flooding spikes.

Payload & Content Behaviour

- **Payload Shannon Entropy ($H_{entropy}$)**: Computed as $H(X) = -\sum p_i \log_2 p_i$. Quantifies data byte randomness in the current packet payload; normal telemetry exhibits low entropy, while random Fuzzy injections exhibit high entropy.
- **Payload Hamming Distance ($H_{distance}$)**: Measures the bitwise difference (Hamming distance) between the current payload and the previous payload of the same CAN ID, capturing anomalous physical state transitions (such as speed or RPM jumps) independent of raw values.

Leakage Prevention A fundamental design decision is to remove features that allow shortcut learning. The following are **never** used as predictive features:

- Raw CAN arbitration IDs
- Raw timestamps
- Attack start positions
- Window indices
- Message indices
- Other positional dataset information

This forces every model to learn behavioural patterns instead of memorizing dataset artifacts.

3.1.4 4. Statistical ML Pipeline

Instead of learning directly from raw CAN traffic, the statistical pipeline operates on the engineered behavioural feature vector extracted from each time window. Each window is represented by timing, payload and communication statistics that summarize the behaviour of the network during that short interval. These features are then supplied independently to multiple tree-based ensemble models, including Random Forest, XGBoost and LightGBM.

Rather than committing to a single model family, Chakravyuham evaluates each architecture independently to understand its detection capability, robustness against dataset artifacts, computational requirements and suitability for deployment on resource-constrained automotive hardware.

Tree-based ensemble methods were selected because they provide several practical advantages for embedded intrusion detection, including fast inference, low memory requirements, straightforward explainability through feature importance analysis, and efficient deployment using compiled inference libraries such as Treelite or m2cgen.

Each trained model produces a window-level prediction indicating whether the observed behaviour corresponds to normal vehicle communication or one of the supported attack categories. The resulting models are then compared using both conventional benchmark metrics and the leakage-aware evaluation methodology developed during this project.

3.1.5 5. Graph Neural Network Pipeline

Instead of treating each CAN message in isolation, the GCN pathway represents every window as a communication graph, allowing the model to learn structural relationships between ECUs rather than just per-message statistics.

Graph Construction

- Each unique CAN arbitration ID within a window becomes a graph node.
- A directed edge is created for every transition between two consecutive CAN messages, representing which ID communicated after which.
- Repeated transitions between the same pair of IDs are accumulated into a single edge weight rather than creating duplicate edges, capturing how strong or frequent that communication pattern is within the window.
- Self-loops are retained to preserve repetitive communication behaviour commonly seen in CAN networks, where the same ID can transmit consecutively.
- If a window happens to produce no repeated transitions between distinct IDs, the graph falls back on self-loops so the convolution still has neighbouring structure to aggregate over.

Node-Level Features

- Each node carries 11 behavioural and statistical descriptors of that arbitration ID's activity within the window (drawn from the traffic, payload, and communication behaviour groups described above), rather than the ID itself.
- Each arbitration ID's typical behaviour is tracked over time, and features are expressed as how much a window deviates from that ID's usual pattern, rather than as raw values. This lets the model spot what looks abnormal for that specific ID, instead of relying on fixed thresholds.

Graph Convolutional Network Architecture

- The graph is passed through two stacked graph convolution layers: GCNConv ($11 \rightarrow 8$) followed by a ReLU activation, then GCNConv ($8 \rightarrow 8$) followed by another ReLU activation.
- Each layer lets a node update its representation by aggregating information from its neighbouring nodes and the edges connecting them, so the model learns structural and relational patterns, such as unusual transition behaviour or abnormal connectivity, rather than relying on any single node's features in isolation.

Pooling

- After the convolution layers, node-level embeddings are aggregated into a single fixed-length graph-level representation using global mean pooling.
- This produces one summary representation per window, combining the behaviour of every ECU present in that window into a form suitable for classification.

Graph-Level Features

- Beyond the pooled node representation, global features describing the overall structure of the window's communication graph are computed separately, such as the number of distinct IDs present, the entropy of transitions across the whole graph, and the overall connectivity density of the graph.

- These global features are only added in after pooling, so the individual ECU behaviour captured by each node and the overall structure of the communication graph are kept separate until the final step, rather than being blended together too early.

Classifier

- Dropout is applied before the final classification layer, randomly deactivating part of the combined representation during training so the model doesn't over-rely on any single feature or pathway, helping prevent overfitting.
- A final linear layer maps this representation to the attack categories being predicted, producing the window's classification.

3.1.6 6. Image-Based Pipeline (MosaicCNN)

To establish a baseline for spatial-temporal representation learning directly on payloads, we implement a Convolutional Neural Network operating on raw CAN sequence images (MosaicCNN):

- **Image Construction (Mosaic Grid):** The 29-bit CAN IDs (optionally compressed via a trained Autoencoder to 9 bits) are combined with byte payloads to form a 32×32 binary matrix window (the "Mosaic").
- **Spatial Convolution:** The binary grid is fed into a 2D CNN with stacked Conv2D, MaxPool2D, and fully connected layers.
- **Comparative Role:** MosaicCNN operates as a spatial pattern baseline, mapping high-frequency payload and ID transitions visually to detect injection anomalies.

3.2 Key Design Philosophy

A core objective throughout Chakravyuham is to ensure that the models learn **behavioural characteristics of CAN traffic rather than memorizing dataset-specific patterns**.

To achieve this, we deliberately exclude features that could act as shortcuts, including raw timestamps, attack start positions, message indices, window positions, and other positional information. Similarly, raw CAN arbitration IDs are never used as predictive features. In the statistical ML pipeline they are transformed into behavioural statistics, while in the GCN they are used only to define graph structure (node identity and communication transitions), never as feature values.

This forces the models to learn **how ECUs communicate, how their behaviour changes over time, and how interactions between ECUs evolve**, instead of simply recognizing identifiers or dataset artifacts. This design philosophy was established after our leakage investigation revealed that models could achieve unrealistically high benchmark performance by exploiting dataset-specific shortcuts rather than genuine behavioural signals. Every subsequent improvement proposed in Chakravyuham is built around preventing shortcut learning and encouraging robust behavioural understanding.

3.3 What Makes This Architecture Different

Unlike conventional benchmark-driven IDS research, Chakravyuham is designed around **deployment-oriented engineering** rather than maximizing dataset accuracy alone.

Leakage-aware evaluation

- Temporal and block-wise dataset partitioning instead of purely random sampling.
- Reduces overly optimistic performance caused by frame-level correlations and dataset leakage.

Behaviour-first feature engineering

- Models learn timing behaviour, payload evolution and communication patterns instead of memorizing identifiers or dataset artifacts.
- Improves robustness and forms the basis for future cross-vehicle generalization.

Comparative architecture evaluation

- Rather than assuming one machine learning model is optimal, Chakravyuham systematically evaluates tree-based ensemble methods and Graph Neural Networks to understand their respective strengths, limitations and deployment feasibility.
- This enables architecture selection to be driven by engineering evidence rather than benchmark popularity.

3.4 Candidate Model Architectures Summary

To provide a rigorous architectural comparison, we benchmark five different models ranging from traditional statistical classifiers to deep neural network pathways. Table 1 outlines their architectural configurations, serialized parameters, and theoretical roles in the defense ecosystem.

Table 1: Comparative Summary of Candidate Model Architectures

Model Class	Algorithm	Memory Size	Avg Latency	Core Structural Role
Tabular ML	XGBoost	0.95 MB	0.38 ms	Bounded-depth GBDT; fast inference on CPU
Tabular ML	LightGBM	1.34 MB	1.18 ms	Leaf-wise GBDT; champion edge classifier
Tabular ML	Random Forest	2.19 MB	2.10 ms	Decision tree bagging; statistical baseline
Graph Neural Net	GCNIDS	0.0033 MB	10.93 ms	Spatial-relational convolution on ECU graphs
Image CNN	MosaicCNN	0.74 MB	1.52 ms	2D spatial convolution on binary packet grids

4 Evaluation Results & Model Benchmarks

We evaluated the candidate models (XGBoost, LightGBM, Random Forest, GCNIDS, and MosaicCNN) across the HCRL (Car-Hacking) and OTIDS datasets under two distinct feature configurations:

1. **Basic Feature Set:** Retains raw arbitration IDs and DLC values as tabular inputs (15 features).
2. **Advanced Feature Set:** Excludes all raw identifiers and DLC values, utilizing only engineered behavioral metrics (frequency, IAT, timing jitter, payload entropy, and payload Hamming distance) to ensure spoofing immunity (5 features).

To ensure realistic performance estimation and prevent temporal proximity leakage, the evaluation was conducted using a strict chronological block-wise temporal split (10-second segment grouping).

Table 2: Model Performance Benchmarks on the HCRL Car-Hacking Dataset

Model	Feature Set	Accuracy (%)	Macro F1 (%)	FPR (%)	Latency (ms)	Size (MB)
XGBoost	Basic	99.9966	99.9904	0.0038	0.401	0.4593
LightGBM	Basic	99.9996	99.9988	0.0004	1.252	0.9055
RandomForest	Basic	99.9725	99.9235	0.0334	1.726	0.7034
GCNIDS	Basic	66.8967	56.9251	0.0184	10.115	0.0033
MosaicCNN	Basic	68.0626	57.7610	0.1726	1.508	0.7375
XGBoost	Advanced	99.9280	99.8312	0.0845	0.422	0.6164
LightGBM	Advanced	99.9959	99.9903	0.0039	0.923	0.8525
RandomForest	Advanced	99.9687	99.9265	0.0369	1.745	1.1600
GCNIDS	Advanced	66.7275	56.7959	0.0084	10.308	0.0033
MosaicCNN	Advanced	66.8304	53.5868	0.6713	1.498	0.7375

Table 3: Model Performance Benchmarks on the OTIDS (KIA Soul) Dataset

Model	Feature Set	Accuracy (%)	Macro F1 (%)	FPR (%)	Latency (ms)	Size (MB)
XGBoost	Basic	87.9923	56.2686	12.8036	0.382	0.5417
LightGBM	Basic	88.9404	64.2521	10.4034	1.186	0.6011
RandomForest	Basic	87.6322	54.4405	13.2490	1.872	1.5301
GCNIDS	Basic	82.3962	54.3648	11.2650	11.280	0.0033
MosaicCNN	Basic	79.4073	38.5910	15.7168	1.480	0.7370
XGBoost	Advanced	87.4272	54.8457	13.2979	0.387	0.6934
LightGBM	Advanced	89.5582	66.3019	10.5303	1.181	0.6814
RandomForest	Advanced	86.7896	50.3871	14.1487	2.102	2.1906
GCNIDS	Advanced	83.9139	60.7933	9.1778	10.937	0.0033
MosaicCNN	Advanced	79.4453	38.6627	15.7565	1.524	0.7370

5 Engineering Challenges & Methodology Evolution

Chakravyuham was not developed as a static pipeline but evolved through multiple iterative stages of experimentation, evaluation, and correction. While initial baselines showed strong benchmark performance on standard evaluation setups, deeper investigation revealed that many of these results were influenced by dataset artifacts, leakage patterns, and shortcut learning rather than true behavioral understanding of CAN bus traffic. This section outlines the key engineering discoveries that shaped the final methodology.

5.1 Initial Baseline and Hidden Optimism

The first version of the system was trained using standard machine learning pipelines on the HCRL Car-Hacking dataset with conventional preprocessing and random frame-level train-test splits. This setup represented the initial developer baseline. Early results showed extremely high performance, with classifications matching or exceeding **99.98% accuracy** and **99.96% Macro F1** scores across attack classes.

However, subsequent investigations revealed that this baseline performance was over-optimistic. Frame-level random shuffling allowed packets from the same millisecond attack burst to be split across training and test sets, resulting in severe temporal leakage. Additionally, TreeSHAP explainability analyses showed that the models were not learning actual anomalous network behaviors but were simply memorizing vehicle-specific CAN arbitration IDs as direct mapping rules. Under out-of-session validation or simulated ID-spoofed adversarial traffic, the baseline models collapsed, indicating that the high accuracy was an artifact of dataset representation rather than genuine intrusion detection capability.

5.2 Discovery of Dataset Leakage and Shortcut Learning

A detailed error analysis revealed that the model was exploiting unintended signals:

- **CAN ID leakage:** certain attack types consistently reused specific arbitration IDs, allowing models to memorize attack identity rather than behavior
- **Temporal proximity leakage:** adjacent frames within the same attack burst were being split across train/test sets, inflating performance
- **Timestamp dependence:** early features inadvertently encoded positional information within logs

This led to a key realization: High accuracy was not necessarily equivalent to true intrusion detection capability. These findings demonstrated that evaluation methodology, rather than model architecture, was the primary factor affecting reported performance, necessitating a redesign of the data handling and training pipeline.

To validate this, we introduced stricter evaluation methods, which immediately exposed significant performance degradation in certain attack categories (especially RPM spoofing), confirming that earlier results were not fully reliable.

5.3 Methodological Corrections

Taking into account critical problems we found such as data leakage and evaluation bias, we progressively redesigned the pipeline around leakage-resistant principles. Rather than optimizing solely for benchmark accuracy, every methodological change was evaluated based on whether it improved the credibility of real-world deployment performance. These are the decisions we came to:

- **Block-wise temporal splitting:** CAN logs were segmented into fixed windows and split chronologically to prevent leakage across adjacent frames
- **Seeded shuffled block partitioning:** ensured all attack classes were fairly represented in train/validation/test splits without introducing temporal overlap
- **Removal of raw positional features:** timestamps, message indices, and session position were excluded as direct features

This shifted the model from learning *where an attack occurs* to *how an attack behaves*.

5.4 Synthetic augmentation + failure insight

A major limitation identified during development was **extreme class imbalance**, where normal traffic dominates the dataset and certain attack types are underrepresented. This resulted in models that were biased toward predicting normal behavior, with weak recall on minority attack classes.

To address this, we experimented with:

- Oversampling underrepresented attack classes
- Synthetic augmentation of attack windows
- Controlled balancing of training distributions

Key finding: Performance improvements were observed only when the model was exposed to a sufficiently balanced representation of attack behaviors. This confirmed that the issue was not model incapability, but **insufficient learning exposure for minority classes**.

However, the experiments also highlighted an important constraint: Blind or uncontrolled synthetic generation can distort the true distribution of CAN bus behavior and degrade model reliability.

5.4.1 Key Constraints for Synthetic Data in CAN IDS

From these experiments, we derived important principles for synthetic data generation:

- Synthetic samples must preserve **temporal structure of attack bursts**, not just individual frames
- Window-level consistency is critical — attacks are behavior sequences, not isolated messages
- Class balance must be improved without breaking natural traffic distribution ratios
- Feature-level distributions (entropy, IAT, frequency) must remain aligned with real attack statistics
- Synthetic augmentation must be constrained within realistic operational ranges to avoid distribution drift

5.4.2 Pipeline Insight: Real + Synthetic Separation Strategy

Based on these findings, we structured the data pipeline conceptually as:

- **Dataset A (Real CAN Traffic)** → Feature extraction → baseline behavioral modeling
- **Dataset B (Synthetic / Augmented Attack Windows)** → Controlled feature generation respecting temporal and statistical constraints
- **Merging Stage** → Feature-level alignment and normalization before training

This reinforced a key insight: Synthetic data is only useful when it reinforces real behavioral structure — not when it attempts to approximate raw signal generation.

5.4.3 Core Conclusion from This Phase

This phase demonstrated that the primary limitation in early models was not architectural or algorithmic, but data exposure imbalance. The model was capable of learning attack behaviour, but required sufficient and well-structured examples of minority attack classes to do so effectively.

This investigation fundamentally changed our perspective on synthetic augmentation. Rather than treating it as simple oversampling, we now view it as a controlled behavioural data generation process that must preserve temporal structure, statistical consistency, and realistic attack dynamics. This directly informs the synthetic augmentation pipeline described in the future roadmap.

5.5 Feature Design Reorientation (From Identity to Behaviour)

One of the most critical changes in the project was redefining feature philosophy. We observed that including raw identifiers (like CAN IDs) as direct features led to:

- Overfitting to ECU identity
- Poor adversarial robustness (especially under spoofing)
- Failure under cross-session evaluation

However, removing them entirely caused:

- Loss of ECU-level behavioral anchoring
- Increased false positives in certain configurations

This led to a controlled compromise:

- CAN IDs were retained only as **graph structure identifiers**, not learning features
- Feature learning was shifted entirely to **behavioral signals**, such as:
 - inter-arrival time statistics
 - entropy-based payload variation
 - transition dynamics between ECUs
 - rolling Z-score deviations from historical baselines

This transition was central in converting the system from a **pattern memorizer to a behavioral anomaly detector**.

5.6 Feature Dropout and Overfitting Control

Further experiments showed that even behavioral models could overfit when feature visibility was unrestricted.

To counter this, we introduced:

- **Feature dropout (column subsampling)** in GBDT models
- Controlled exposure of CAN ID-related structure during training
- Constraints on tree depth and feature influence

This created a balance between:

- learning ECU identity relationships (useful for detection)

6 Industry Landscape & Remaining Challenges

6.1 Commercial Solutions

The automotive cybersecurity landscape is currently dominated by a small number of Tier-1 security vendors, primarily operating under regulatory constraints such as **UN R155** and **ISO/SAE 21434**, which mandate intrusion detection, incident response, and fleet-level monitoring.

Bosch ETAS (ESCRYPT) is one of the most widely deployed solutions:

- **CycurIDS-CAN** performs CAN intrusion detection using deterministic, rule-based logic generated from OEM vehicle network specifications.
- **CycurIDS-M** aggregates detected security events into a centralized Security Event Management system for fleet-level analysis.
- **CycurGATE** extends protection to automotive Ethernet through hardware-accelerated firewalling and packet inspection at line speed.

PlaxidityX (Argus Cyber Security) implements a Vehicle Detection and Response (VDR) framework:

- Software-based IDPS for CAN intrusion detection using timing and behavioral rules
- Gateway/ECU-level deployment for in-vehicle monitoring
- Hardware-based **vDome**, which uses ECU electrical fingerprinting for physical-layer attack detection

6.2 Current Industry Approach

Across both companies, the dominant design pattern is:

- **Rule-based detection on vehicle**
- **Signature/heuristic systems for known behaviors**
- **Cloud / VSOC-based analytics for fleet intelligence**

Machine learning is generally not deployed as the primary in-vehicle detection mechanism.

6.3 Why the Industry Avoids Heavy ML On-Vehicle

This design choice is driven by practical automotive constraints:

- **Deterministic execution requirements** (predictable behavior required for safety certification under ISO 26262)
- **Limited ECU compute and memory budgets**
- **Real-time latency constraints**
- **Availability of OEM network specifications enabling rule generation**
- **Hard problem of cross-vehicle generalization (no standard representation across OEMs)**

As a result, ML is often reserved for **off-vehicle analytics (VSOC / cloud)** rather than real-time in-vehicle detection.

6.4 Remaining Gaps in Existing Systems

Despite maturity in rule-based systems, several key limitations remain:

- **Behavioral attacks are hard to generalize with rules** (subtle timing drift, impersonation patterns)
- **Limited cross-vehicle generalization** due to OEM-specific configurations
- **High dependency on manual rule engineering per vehicle model**
- **Weak handling of unseen attack variations**
- **Evaluation bias due to dataset artifacts in ML-based research systems**
- **No unified, scalable learning-based representation of CAN behavior**

This creates a clear gap between:

- Highly deployable but rigid rule-based systems
- vs
- Flexible but non-deployable ML-based research systems

6.5 Where Chakravyuham Fits

Chakravyuham is positioned at the intersection of these two paradigms:

- Retains **deterministic deployability principles** (edge-feasible, low-latency models)

- Introduces **behavioral machine learning over rule systems**
- Moves away from raw CAN ID dependence toward **structured behavioral representation**
- Designs for **deployment realism rather than benchmark optimization**

This directly addresses the gap between industrial rule-based systems and academic ML-based IDS models by focusing on:

- Real-time feasibility
- Cross-vehicle adaptability (via structured representations like VSS directionally)
- Robust behavioral learning instead of dataset memorization

7 Future Roadmap

This roadmap is derived directly from limitations observed during model development, evaluation, and adversarial testing. Each component targets a specific failure mode identified in dataset behavior, model generalization, or deployment constraints.

Rather than adding standalone features, this section outlines the **engineering steps required to transition from a high-performing prototype to a deployment-ready automotive intrusion detection system.**

7.1 Dataset Dependence → Synthetic Attack Augmentation Engine

A key limitation in IDS research, including our experiments, is that high benchmark performance often results from **dataset-specific artifacts rather than true behavioral learning**. In structured datasets like HCRL, attack patterns can contain exploitable shortcuts that models memorize instead of generalizing behavior.

Proposed Solution We introduce a synthetic augmentation framework that:

- Generates controlled variations of:
 - DoS flooding patterns
 - Fuzzy injection behavior
 - Impersonation-style payload drift
- Preserves CAN temporal integrity:
 - message ordering consistency
 - inter-arrival timing structure
 - per-ID communication dynamics
- Balances underrepresented attack classes to reduce skewed learning

Objective Replace dataset shortcuts with **behaviorally diverse attack representations**, improving robustness beyond benchmark conditions.

7.2 OEM Dependence → Standardized Signal Abstraction (VSS-Driven, Conditional)

A major barrier in real-world deployment is that CAN-based IDS models are tightly coupled to **OEM-specific CAN IDs and message formats**, limiting cross-vehicle transferability.

Proposed Solution We were inspired by the growing industry adoption of the **COVESA Vehicle Signal Specification (VSS)** and chose to adopt it as a normalization layer:

- Maps raw CAN IDs to standardized signal representations
- Enables learning over signal semantics instead of vendor-specific identifiers
- Allows architecture reuse across OEMs without redesign

Feasibility Constraint

- If DBC / OEM mappings are available → full VSS pipeline is implementable
- Otherwise → treated as structured future integration with OEM collaboration

Objective Enable **cross-vehicle portability** through **standardized representation**

7.3 Cross-Vehicle Generalization → Behavioral Calibration Layer

Even with standardized representations, vehicles differ significantly in real-world behavior:

- ECU timing profiles vary across models
- traffic frequency distributions differ by vehicle type
- operational conditions introduce behavioral drift

Proposed Solution We introduce a behavioral calibration layer that:

- Maintains per-vehicle rolling baseline statistics
- Computes deviation-based features (relative behavioral modeling)
- Stores lightweight vehicle profiles (e.g., `vehicle.yaml`) capturing:
 - frequency distributions
 - timing baselines
 - entropy ranges

Objective Shift detection from **absolute thresholds** → **behavioral deviations**, enabling robust cross-vehicle generalization.

7.4 Per-OEM Calibration Model → Industry-Aligned Adaptation Strategy

Instead of a universal model, we align with real automotive deployment practices.

Proposed Solution Drawing inspiration from leaders in the industry such as ETAS, we chose to adopt the follow approach rather than follow a one-size-fits-all approach:

- A pretrained base model trained on diverse datasets
- Lightweight per-OEM calibration using:
 - DBC-derived statistics
 - vehicle behavior profiles
- No full retraining required per vehicle

Objective Enable **fast onboarding with consistent detection architecture**

7.5 Hybrid Rule + ML Detection Layer → Deployment-Ready System

Real automotive IDS systems require deterministic safeguards alongside statistical models.

Proposed Design Rather than replacing existing automotive security practices, our long-term objective is to complement them. The proposed architecture follows the defence-in-depth philosophy already adopted by commercial platforms such as ETAS CycurIDS and PlaxidityX, combining deterministic rule-based filtering with behavioural machine learning for more adaptive intrusion detection.

Rule-based layer (fast filtering):

- DLC validation
- Known-bad ID and rate thresholds
- Protocol sanity checks

ML-based layer (behavioral detection):

- XGBoost / LightGBM for structured features
- GCN-based model for structural behavior learning (research branch)

Objective Combine:

- deterministic safety guarantees
- adaptive ML-based anomaly detection

for reliable deployment feasibility.

7.6 Edge Deployment and Lightweight Inference Pipeline

Automotive in-vehicle systems operate under strict real-time execution constraints and extremely limited memory limits. To demonstrate the real-world deployability of the Chakravyuham detection system on Electronic Control Units (ECUs), we benchmarked the models on constrained edge configurations:

- **LightGBM**: Serialized model size of **1.34 MB** (reducing to **0.68 MB - 0.85 MB** under downsampled configurations) with an average inference latency of **0.92 ms - 1.18 ms** per window.
- **XGBoost**: Serialized model size of **0.95 MB** (reducing to **0.38 MB - 0.69 MB** under downsampled configurations) with a sub-millisecond inference latency of **0.38 ms**.
- **GCNIDS**: Serialized weights size of **0.0033 MB** (extremely compact) with an average inference latency of **10.3 ms** (constrained by graph-construction overhead on low-cost hardware).
- **MosaicCNN**: Serialized size of **0.74 MB** with a latency of **1.52 ms**.

Proposed Solution

- Tree models compiled directly to optimized C code using compiled libraries (Treelite / m2cgen) to eliminate interpreter and runtime overhead.
- Bounded-memory rolling circular arrays for real-time windowing and z-score baseline updates.
- Real-time inference optimization on embedded microcontrollers.

Prototype Scope

- Raspberry Pi 4 and NVIDIA Jetson Nano edge benchmarking.
- Validation of CPU usage, execution jitter, and heap allocations under continuous CAN traffic simulation.

Objective Ensure **real-time feasibility under embedded ECU constraints**

7.7 Replay Attack Detection

The current prototype focuses on DoS, Fuzzy, and Impersonation attacks, which are well represented in the datasets used for development. Replay attacks require reasoning over long-term message sequences rather than short behavioral windows. Traditional ML models typically aren't capable of capturing the relevant attack patterns for such attacks. As future work, we plan to evaluate sequence-aware deep learning architectures (e.g., LSTMs, Temporal GNNs, or Transformers) for replay attack detection while investigating whether their computational requirements remain compatible with edge deployment constraints.

7.8 VSOC-Lite Feedback Loop → Controlled Model Evolution

IDS systems must evolve safely over time without risking system integrity.

Proposed Solution

- Local anomaly aggregation layer (VSOC-lite)
- Structured event logging pipeline
- Central validation before model updates
- OTA updates as future extension

Objective Enable **safe, validated model evolution without on-device retraining**

7.9 Deployment Across Vehicle Architectures

The system is designed for both current and future automotive architectures.

Legacy vehicles

- Gateway ECU-based deployment
- Central CAN traffic aggregation

Zonal architectures

- Deployment shifts to zonal controllers
- Enables earlier detection closer to signal source

Objective Ensure compatibility across **legacy and next-generation vehicle systems**

7.10 Physical-Layer Attack Coverage (Defined Scope Boundary)

The system focuses on **network-level intrusion detection at the gateway layer**.

It does NOT directly address:

- pre-gateway physical-layer CAN injection attacks

These are typically handled by complementary systems such as:

- ECU-level IDPS
- hardware fingerprinting solutions (e.g., PlaxidityX vDome)

Objective Clearly define system boundaries while maintaining compatibility with full-stack automotive security ecosystems.

7.11 Final Note

This roadmap transforms Chakravyuham from a dataset-trained prototype into a **deployment-aware automotive cybersecurity framework**, addressing:

- dataset bias
- OEM-specific coupling
- cross-vehicle generalization
- real-time constraints
- system-level scalability

Each component directly maps to a known limitation in either academic IDS research or industrial automotive security systems.

8 Tech stack

To facilitate rapid prototyping, validation, and dashboard visualization, we selected an enterprise-grade technology stack outlined in Table 4.

Acknowledgments

This work was developed as a technical concept proposal for the Tata Technologies InnoVent-27 (4th Edition) hackathon under the theme “AI at the Edge” for the Automotive Sector. We express our gratitude to Tata Technologies, Amazon Web Services (AWS), and Emerson’s Test & Measurement business for hosting this platform to foster edge-feasible real-time intelligence solutions for vehicle cybersecurity.

9 References

9.1 Academic References

- HCRL Car-Hacking Dataset, <https://ocslab.hksecurity.net/Datasets/car-hacking-dataset>
- HCRL CAN Dataset for intrusion detection (OTIDS), <https://ocslab.hksecurity.net/Dataset/CAN-intrusion-dataset>
- GCNIDS: Graph Convolutional Network based Intrusion Detection System for In-Vehicle Networks, <https://arxiv.org/abs/2309.10173>

Table 4: Chakravyuham Development and Operations Technology Stack

Category	Tooling / Libraries	Implementation Role
Programming	Python, TypeScript, Node.js, Vite	Core backend research, FastAPI service, and React 19 dashboard
Data Handling	Pandas, NumPy	Ingestion of raw logs, rolling statistics, and feature normalization
Machine Learning	Scikit-learn, XGBoost, LightGBM	Statistical GBDT classification and latency evaluation
Graph Networks	PyTorch, PyTorch Geometric (PyG)	2D GCN spatial convolution and topological routing models
Explainability	SHAP (TreeSHAP)	Anomaly contribution analysis and shortcut memorization checks
Edge Deployment	ONNX, Treelite, m2cgen	Lightweight serialization and tree-to-C compilation targets
Dashboard	FastAPI, Tailwind CSS, Framer Motion	Live dashboard log monitoring, charts, and telemetry averages
Datasets	HCRL (Car-Hacking), OTIDS (KIA Soul)	Raw in-vehicle attack captures and cross-validation logs

- Jeep Cherokee Remote Exploit (Miller & Valasek, 2015), <https://ieeexplore.ieee.org/document/8890036>
- UN Regulation R155, <https://unece.org/transport/documents/2021/03/standards/un-regulation-no-155-cyber-security-and-cyber-security>
- ISO/SAE 21434 Road Vehicles — Cybersecurity Engineering, <https://www.iso.org/standard/70918.html>
- Intrusion Detection in Vehicle Controller Area Network (CAN) Bus Using Machine Learning: A Comparative Performance Study, https://www.researchgate.net/publication/369725497_Intrusion_Detection_in_Vehicle_Controller_Area_Network_CAN_Bus_Using_Machine_Learning_A_Comparative_Performance_Study
- A Survey of Intrusion Detection for In-Vehicle Networks, https://www.researchgate.net/publication/334821425_A_Survey_of_Intrusion_Detection_for_In-Vehicle_Networks
- CAN-D: A CAN Decoder for Automotive Cybersecurity (IEEE Transactions on Dependable and Secure Computing, 2020), <https://ieeexplore.ieee.org/document/9099616>
- ACTIDS: Automotive Controller Area Network Intrusion Detection System (IEEE Internet of Things Journal, 2021), <https://ieeexplore.ieee.org/document/9459462>
- A Deep Learning Approach for CAN Bus Intrusion Detection (IEEE Transactions on Vehicular Technology, 2018), <https://ieeexplore.ieee.org/document/8357490>

9.2 Industry References

- Bosch ETAS — CycurIDS-CAN, CycurGATE and Vehicle Security Operations Center (VSOC), <https://www.etas.com/ww/en/products-services/cybersecurity-products/escrypt-intrusion-detection-and-prevention-solution/>
- PlaxidityX (formerly Argus Cyber Security), <https://plaxidityx.com/>, <https://plaxidityx.com/products/vehicle-cyber-protection-system/>
- COVESA Vehicle Signal Specification (VSS), <https://covesa.global/project/vehicle-signal-specification/>
- AWS IoT FleetWise, <https://aws.amazon.com/iot-fleetwise/>
- AUTOSAR Intrusion Detection System Manager (IdsM), https://www.autosar.org/fileadmin/standards/R20-11/CP/AUTOSAR_SWS_IntrusionDetectionSystemManager.pdf